

Proposed System Architecture for “GroundTruth” Claim Tracker

Data Ingestion & Claim Extraction

Gather the corpus (e.g. scientific articles) and preprocess by semantic chunking. Use LLM pipelines to break each document into verifiable facts. For example, Microsoft’s “Claimify” approach first extracts simple factual statements from text for independent verification [6†L28-L37]. In practice, an LLM (GPT-4o or open model) can be prompted (possibly with chain-of-thought) to parse each chunk into claim triples (subject–predicate–object). Research like CoDe-KG demonstrates end-to-end pipelines where LLMs split complex sentences (with coreference resolution) into dozens of high-precision triples [17†L77-L85]. Similarly, fact-checking frameworks such as GraphCheck use LLMs to transform claims into structured entity–relationship graphs before verification [12†L139-L147]. In our system, each identified claim is tagged with its source (document ID, authors) and timestamp (publication date).

Knowledge Graph & Data Stores

Store claims and their provenance in a **graph database**. Nodes represent entities (claims, concepts, authors, publications) and edges encode relationships (“claims X implies Y”, “source_of”, “refutes”, etc.). For example, ArangoDB’s GraphRAG approach treats data as nodes/edges so that retrieval traverses the graph contextually [40†L295-L303]. Neo4j’s LLM-based builder similarly ingests chunks and creates “GraphDocuments” of nodes and relationships [42†L268-L275]. Graph systems (Neo4j, ArangoDB, TigerGraph, etc.) let you query multi-hop chains of evidence (e.g. “which claim was later contradicted by which author?”). Many graph databases support **hybrid search**: embedding and full-text. For instance, Neo4j’s pipeline updates chunk similarities via KNN on embeddings, and builds combined vector+keyword indexes for fast semantic search [42†L321-L329]. In practice you may choose a **multi-model DB** (e.g. ArangoDB) that also stores raw text (JSON docs) and supports full-text indexing alongside graph data [40†L320-L329].

To complement the graph, maintain a **vector index** of claim embeddings. Vector DBs (e.g. Milvus, Weaviate, Pinecone, FAISS) let you embed each claim (using SBERT or OpenAI embedding models) for nearest-neighbor search. A vector database “stores, indexes, and queries high-dimensional vector data” optimized for cosine or KNN similarity [46†L21-L28]. This enables fast retrieval of semantically similar claims (e.g. different papers stating the same hypothesis). You can then link these in the graph (or use the DB for quick lookups).

Tech Examples: Use Neo4j or Arango for the knowledge graph; use Weaviate or Milvus for embeddings. Neo4j’s [LLMGraphTransformer] can automatically map extracted text into a graph [42†L268-L275]. If cost is a concern, open-source hosts or managed clouds (Neo4j Aura, ArangoDB Oasis) are options.

Compression: Because LLMs and embeddings are cheaper than before, you can store vector representations of large text chunks and only load full text on demand.

Embedding Models & LLMs

Large language models serve two roles: extracting claims and reasoning over them. Use a powerful LLM (GPT-4 family, Claude, or a tuned Llama3) to parse and decompose text. For example, in the pipeline above, an LLM is prompted to identify sub-claims and entities. Then use lighter-weight embedding models to index and compare claims. OpenAI's **text-embedding-3** or open models like SentenceTransformers can embed each claim sentence. These embeddings power both the vector store and similarity ranking for new queries. As Neo4j demonstrated, you can embed entities and use KNN to reinforce graph edges [42†L321-L329] . In summary:

- **LLMs (GPT-4o, Llama3, etc.)** – to chunk text and generate claim triples.
- **Embeddings (SBERT, OpenAI)** – to index claims and compute similarity for retrieval and clustering.
- **Specialized tasks:** you might fine-tune a small model on claim detection if needed.

These models enable continuous learning: as new papers arrive, a “Temporal Agent” pipeline (see below) can re-run extraction and update graphs.

Temporal Reasoning & Updates

Unlike static KGs, we need **time-aware facts**. Attach timestamps (e.g. publication date) to each claim or graph edge. Build a *temporal knowledge graph* so you can ask “what was the consensus in 2010 vs today?” OpenAI’s “Temporal Agent” approach exemplifies this: raw documents are chunked into statements, then converted into **time-stamped triplets**, and an “Invalidation Agent” flags any earlier claims invalidated by later data [48†L951-L960] . In practice, maintain for each claim-node a `valid_from` (and optionally `valid_until`) date. As new evidence arrives, run an invalidation check: e.g. mark old claims as superseded when a newer source contradicts them. This ensures the graph can answer *when* a claim became outdated. Temporal graphs allow precise queries like “find claims from 2005-2015 now disproven by later research” [48†L951-L960] .

System Flow Diagram

[37†embed_image] *Example architecture blueprint.* Data enters from publications, is chunked and passed through an LLM extraction service. Claims and entities are inserted into a graph DB (with timestamp metadata), and claim embeddings go to a vector store. A query service performs hybrid search: it retrieves candidates via vector similarity and traverses graph edges for supporting evidence. The graph and vector stores update continuously as new papers are ingested, using an asynchronous pipeline (e.g. Kafka/Airflow orchestration) to parallelize extraction, embedding, and indexing.

Tech Stack & Deployment

Backend: Python or Java services using NLP libraries (spaCy for NER, LangChain’s LLMGraphTransformer [42†L268-L275] , etc.). For orchestration, Airflow/Kafka stream for ETL. **LLM/Embedding:** OpenAI API (GPT-4o, text-embedding-3) or open models (Llama3, SentenceTransformer). **Databases:** Neo4j or ArangoDB for the knowledge graph (providing schema flexibility and hybrid RAG support [40†L295-L303] [42†L268-L275]); a vector store like Weaviate/Milvus for semantic search. **Infrastructure:** Containerize with Docker/Kubernetes; cloud-hosted DBs for scalability. **Cost/Scaling:** Graph-based RAG is shown to

deliver richer, more accurate retrieval than vector-only solutions [46†L67-L70] . By compressing text into vectors and triples, and archiving low-value edges (OpenAI suggests pruning unused nodes [48†L1008-L1016]), the system stays efficient.

Summary: This pipeline enables automated claim provenance: ingest documents, extract claims via LLMs, store them in a time-aware graph, and use embeddings for fast retrieval. Such a design leverages modern LLM and graph-tech to reveal how ideas evolve over time, answering queries like “Which widely-cited claim was later refuted by the literature?” [48†L951-L960] [42†L268-L275] .

Sources: State-of-the-art techniques (LLM-based KG extraction, vector search, temporal KGs) from recent research and tech blogs were used to inform this architecture [6†L28-L37] [40†L295-L303] [42†L268-L275] [48†L951-L960] .
